

Tab 1

FINAL 30-WEEK MAANG EXECUTION SCHEDULE

Total Duration: 30 Weeks (7.5 Months)

Daily Time: 4–6 hours

Total Problems: ~430 problems (deep mastery)

Outcome: Top 1-3% DSA capability → MAANG offers



COMPLETE WEEK-BY-WEEK BREAKDOWN

PHASE 1: FOUNDATION (Weeks 0-6)

WEEK 0 — Arrays + Sorting Fundamentals

Topics:

- Kadane's algorithm
- Dutch National Flag
- Quick Select
- Merge Sort, Quick Sort
- Custom comparators - master Java Streams operation and come back
- In-place operations

Problems (15):

1. Maximum Subarray (Kadane) ★
2. Product of Array Except Self ★
3. Next Permutation
4. Rotate Array
5. Find Duplicate Number
6. Set Matrix Zeroes
7. Spiral Matrix
8. Sort Colors (Dutch National Flag) ★
9. Kth Largest Element (QuickSelect) ★
10. Merge Sorted Array
11. Sort an Array (implement QuickSort)
12. Sort an Array (implement MergeSort)
13. 3-Way QuickSort
14. Maximum Product Subarray
15. Best Time to Buy and Sell Stock

Revision:

- Day 6: Kadane, Sort Colors, QuickSelect
 - Day 7: Pattern summary + complexity analysis
-

WEEK 1 — Sliding Window + Two Pointers

Topics:

- Fixed window
- Variable window
- At most K / exactly K
- Fast-slow pointer
- Left-right pointers
- Partition logic

Problems (15):

1. Longest Substring Without Repeating Characters ★
2. Longest Substring with At Most K Distinct Characters ★
3. Minimum Window Substring ★
4. Maximum Sum Subarray of Size K
5. Minimum Size Subarray Sum
6. Subarray Product Less Than K
7. Trapping Rain Water ★
8. Container With Most Water ★
9. Remove Duplicates from Sorted Array
10. Remove Duplicates from Sorted Array II
11. Move Zeroes
12. Squares of Sorted Array
13. 3Sum Closest
14. Subarrays with K Different Integers
15. Fruit Into Baskets

Revision:

- Day 6: Re-solve Minimum Window, Trapping Rain Water
 - Day 7: Identify window type in 5 new problems (don't solve)
-

WEEK 2 — Hashing + Prefix Sum + String Manipulation

Topics:

- Frequency counting
- Prefix sums
- Hash-based detection

- String character manipulation
- Anagram patterns

Problems (20):

1. Two Sum ★
2. 3Sum ★
3. 4Sum
4. Group Anagrams ★
5. Valid Anagram
6. Subarray Sum Equals K ★
7. Longest Consecutive Sequence ★
8. Continuous Subarray Sum
9. Max Points on a Line
10. Subarrays with K Different Integers
11. Longest Palindromic Substring ★
12. Palindrome Permutation
13. Isomorphic Strings
14. Word Pattern
15. String to Integer (atoi)
16. Zigzag Conversion
17. Find All Anagrams in String
18. Minimum Window Substring (revisit)
19. Longest Repeating Character Replacement
20. Permutation in String

Revision:

- Re-solve: Two Sum, Subarray Sum Equals K, Longest Consecutive Sequence
-

WEEK 3 — Linked List Patterns

Topics:

- Fast-slow pointers
- Reversal techniques
- Merge patterns
- Dummy node technique

Problems (20):

1. Reverse Linked List ★
2. Reverse Linked List II ★
3. Reverse Nodes in K-Group ★
4. Linked List Cycle ★
5. Linked List Cycle II ★
6. Middle of Linked List

7. Palindrome Linked List ★
8. Merge Two Sorted Lists ★
9. Merge K Sorted Lists ★
10. Sort List ★
11. Add Two Numbers ★
12. Add Two Numbers II
13. Remove Nth Node From End
14. Intersection of Two Linked Lists
15. Reorder List ★
16. Swap Nodes in Pairs
17. Odd Even Linked List
18. Partition List
19. Remove Duplicates from Sorted List
20. Remove Duplicates from Sorted List II

Revision:

- Reverse K-Group, Merge K Lists, Reorder List
-

WEEK 4 — Stack Patterns (All 5 Types)

Topics:

- Matching stack
- Monotonic stack
- Boundary stack
- Index stack
- Context stack

Problems (25):

1. Valid Parentheses ★
2. Longest Valid Parentheses ★
3. Minimum Add to Make Parentheses Valid
4. Score of Parentheses
5. Daily Temperatures ★
6. Next Greater Element I ★
7. Next Greater Element II
8. Next Greater Element III
9. Largest Rectangle in Histogram ★★★★★
10. Trapping Rain Water (stack approach) ★
11. Stock Span Problem
12. Sum of Subarray Minimums ★
13. Sum of Subarray Ranges
14. Remove K Digits
15. Decode String ★★
16. Basic Calculator ★

17. Basic Calculator II ★
18. Basic Calculator III
19. Evaluate Reverse Polish Notation
20. Simplify Path
21. Remove All Adjacent Duplicates
22. Remove All Adjacent Duplicates II
23. Asteroid Collision
24. 132 Pattern
25. Online Stock Span

Revision:

- Histogram, Decode String, Longest Valid Parentheses, Basic Calculator
-

WEEK 5 — Heap / Priority Queue

Topics:

- Min heap / Max heap
- Top K elements pattern
- Multi-way merge pattern
- Lazy removal technique

Problems (15):

1. Kth Largest Element in Array ★
2. Top K Frequent Elements ★
3. K Closest Points to Origin ★
4. Kth Smallest Element in Sorted Matrix
5. Find K Pairs with Smallest Sums
6. Task Scheduler ★
7. IPO
8. Reorganize String ★
9. Rearrange String K Distance Apart
10. Merge K Sorted Lists (heap approach) ★
11. Smallest Range Covering Elements from K Lists ★
12. Find Median from Data Stream ★
13. Sliding Window Median
14. Meeting Rooms II (heap approach)
15. Ugly Number II

Revision:

- Kth Largest, Merge K Lists, Task Scheduler
-

WEEK 6 — Binary Search Mastery

Topics:

- Classical binary search
- Search in rotated arrays
- Binary search on answer (monotonic predicate)
- Search space reduction

Problems (20):

1. Binary Search ★
2. First Bad Version
3. Search Insert Position
4. Find First and Last Position ★
5. Search in Rotated Sorted Array ★
6. Search in Rotated Sorted Array II
7. Find Minimum in Rotated Sorted Array ★
8. Find Minimum in Rotated Sorted Array II
9. Find Peak Element ★
10. Koko Eating Bananas ★★
11. Minimum Days to Make M Bouquets
12. Capacity to Ship Packages Within D Days ★★★
13. Split Array Largest Sum ★★
14. Allocate Books (GFG)
15. Aggressive Cows (GFG)
16. Median of Two Sorted Arrays ★★★★★
17. Find K Closest Elements
18. Sqrt(x)
19. Valid Perfect Square
20. Minimize Max Distance to Gas Station

Revision:

- Koko, Ship Packages, Rotated Array, Median of Two Sorted



WEEK 0.5/1 — MATRIX/2D ARRAYS MASTERY

Duration: 7 days

Problems: 20-25

Goal: Complete matrix pattern mastery before advanced topics

Topics Covered:

Day 1-2: Matrix Traversal Fundamentals

- Row-wise, column-wise traversal
- Diagonal traversal

- Spiral traversal
- Zigzag traversal
- Snake pattern
- Boundary traversal

Day 3-4: Matrix Manipulation

- Rotate matrix (90°, 180°, 270°)
- Transpose matrix
- In-place modifications
- Set zeroes
- Fill techniques

Day 5: Matrix Search

- Search in row-wise sorted matrix
- Search in row-column sorted matrix
- Binary search on matrix
- Peak element in 2D

Day 6: Matrix Math & Properties

- Matrix multiplication
- Determinant (if needed)
- Prefix sum 2D
- Range sum query 2D

Day 7: Mixed Practice & Revision

- Hardest problems from week
- Pattern recognition drill
- Timed practice

Complete Problem List (25 Problems):

A) Traversal Patterns (7 problems)

1. **Spiral Matrix** ★★
 - Pattern: Layer-by-layer traversal
 - Key: Boundary management
2. **Spiral Matrix II** ★
 - Generate spiral matrix
 - Key: Direction control
3. **Diagonal Traverse** ★★
 - Diagonal zigzag pattern
 - Key: Direction changes on boundaries
4. **Print Matrix in Snake Pattern** (GFG)

- Alternate left-right, right-left
 - Key: Row parity
 - 5. **Print Matrix in Diagonal Pattern** (GFG)
 - All diagonals sequentially
 - Key: Sum of indices
 - 6. **Boundary Traversal of Matrix** (GFG)
 - Print outer boundary
 - Key: Edge handling
 - 7. **Zigzag (Wave) Traversal** (GFG)
 - Column-wise zigzag
 - Key: Column parity
-

B) Matrix Rotation & Transformation (6 problems)

- 8. **Rotate Image (90° clockwise)** ★★
 - In-place rotation
 - Key: Transpose + reverse rows
 - **CRITICAL PATTERN**
 - 9. **Rotate Image 180°** (variation)
 - Two rotations OR reverse rows + columns
 - 10. **Transpose Matrix**
 - Convert rows to columns
 - Key: Swap $[i][j]$ with $[j][i]$
 - 11. **Set Matrix Zeroes** ★★
 - Set entire row/col to zero
 - Key: In-place marking using first row/col
 - **AMAZON FAVORITE**
 - 12. **Reshape Matrix** ★
 - Convert $m \times n$ to $r \times c$
 - Key: Linear mapping
 - 13. **Flip Image** ★
 - Horizontal flip + invert
 - Key: Two-pointer in each row
-

C) Matrix Search (5 problems)

- 14. **Search a 2D Matrix** ★★
 - Sorted matrix (treat as 1D array)
 - Key: Binary search with row $\times$ col conversion
 - **GOOGLE FAVORITE**
- 15. **Search a 2D Matrix II** ★★
 - Row-wise & column-wise sorted
 - Key: Start from top-right or bottom-left
 - **META FAVORITE**
- 16. **Find Peak Element II** ★★

- 2D peak (greater than all 4 neighbors)
 - Key: Binary search on columns
 - 17. **Kth Smallest Element in Sorted Matrix** ★★
 - Row & column sorted
 - Key: Binary search on value range OR min-heap
 - 18. **Count Negative Numbers in Sorted Matrix** ★
 - Sorted rows & columns
 - Key: Staircase search
-

D) Matrix Math & Prefix Sum (4 problems)

- 19. **Range Sum Query 2D - Immutable** ★★
 - Submatrix sum queries
 - Key: 2D prefix sum
 - **CRITICAL PATTERN FOR DP**
 - 20. **Range Sum Query 2D - Mutable** ★★
 - With updates
 - Key: 2D Binary Indexed Tree (advanced)
 - 21. **Matrix Block Sum** ★
 - Sum of submatrix around each cell
 - Key: 2D prefix sum
 - 22. **Valid Sudoku** ★★
 - Check sudoku validity
 - Key: Row/col/box validation
-

E) Matrix Island Patterns (Preview for Week 11) (3 problems)

- 23. **Island Perimeter** ★
 - Count perimeter of island
 - Key: Count edges
 - 24. **Max Area of Island** ★★
 - Find largest island
 - Key: DFS with area counting
 - **Preview of matrix-as-graph**
 - 25. **Number of Closed Islands** ★★
 - Islands not touching boundary
 - Key: DFS with boundary check
-

Learning Outcomes:

After this week, you will:

- ✓ Master all matrix traversal patterns
 - ✓ Solve rotation/transformation in-place instantly
 - ✓ Recognize when to use 2D prefix sum
 - ✓ Know staircase search pattern for sorted matrices
 - ✓ Convert matrix indices $\leftrightarrow$ linear indices fluently
 - ✓ Handle boundary conditions without bugs
 - ✓ Build foundation for matrix-as-graph (Week 11)
 - ✓ Build foundation for 2D DP (Week 19)
-

PHASE 2: TREES + BACKTRACKING (Weeks 7-10)

WEEK 7 — Tree DFS + BFS Traversals

Topics:

- Inorder, preorder, postorder
- Level-order (BFS)
- Iterative traversals
- Morris traversal
- Tree properties

Problems (20):

1. Binary Tree Inorder Traversal ★
2. Binary Tree Preorder Traversal ★
3. Binary Tree Postorder Traversal ★
4. Binary Tree Level Order Traversal ★
5. Binary Tree Zigzag Level Order ★
6. Binary Tree Right Side View ★
7. Binary Tree Left Side View
8. Vertical Order Traversal ★
9. Boundary of Binary Tree
10. Diagonal Traversal
11. Maximum Depth of Binary Tree
12. Minimum Depth of Binary Tree
13. Same Tree
14. Symmetric Tree ★
15. Invert Binary Tree ★
16. Merge Two Binary Trees
17. Flatten Binary Tree to Linked List ★
18. Average of Levels
19. Maximum Width of Binary Tree
20. Populating Next Right Pointers ★

Revision:

- All 4 traversals (iterative), Zigzag, Vertical Order
-

WEEK 8 — Path Recursion + Bottom-Up Tree DP

Topics:

- Root-to-leaf paths
- Any-to-any paths
- Path sum variations
- Bottom-up aggregation
- Tree DP

Problems (20):

1. Binary Tree Paths
2. Path Sum ★
3. Path Sum II ★
4. Path Sum III ★
5. Sum Root to Leaf Numbers ★
6. Diameter of Binary Tree ★★
7. Binary Tree Maximum Path Sum ★★★★★
8. Longest Univalue Path
9. Lowest Common Ancestor ★★
10. Lowest Common Ancestor of Deepest Leaves
11. House Robber III ★★
12. Binary Tree Cameras ★★
13. Distribute Coins in Binary Tree ★
14. Balanced Binary Tree ★
15. Maximum Product of Splitted Binary Tree
16. Count Good Nodes in Binary Tree
17. Pseudo-Palindromic Paths
18. All Nodes Distance K ★
19. Sum of Distances in Tree
20. Delete Nodes and Return Forest

Revision:

- Diameter, Max Path Sum, LCA, House Robber III
-

WEEK 9 — BST Patterns

Topics:

- BST properties

- BST validation
- BST operations
- Inorder successor/predecessor
- BST construction

Problems (15):

1. Validate Binary Search Tree ★★
2. Kth Smallest Element in BST ★
3. Inorder Successor in BST ★
4. Delete Node in BST ★
5. Insert into BST
6. Search in BST
7. Convert Sorted Array to BST ★
8. Convert Sorted List to BST
9. Recover Binary Search Tree ★★
10. Two Sum IV - Input is BST
11. Serialize and Deserialize BST
12. Balance a BST
13. Largest BST Subtree ★
14. Unique Binary Search Trees ★
15. Unique Binary Search Trees II

Revision:

- Validate BST, Kth Smallest, Delete Node, Recover BST
-

WEEK 10 — Backtracking (Complete)

Topics:

- Subsets pattern
- Combinations pattern
- Permutations pattern
- Constraint satisfaction
- Pruning techniques

Problems (23):

Subsets/Combinations:

1. Subsets ★
2. Subsets II ★
3. Combination Sum ★
4. Combination Sum II ★
5. Combination Sum III
6. Combination Sum IV
7. Palindrome Partitioning ★★

8. Generate Parentheses ★★
9. Letter Combinations of a Phone Number ★
10. Restore IP Addresses ★

Permutations: 11. Permutations ★ 12. Permutations II ★ 13. Next Permutation (revisit)
14. Palindrome Permutation II

Constraint Satisfaction: 15. N-Queens ★★ 16. N-Queens II 17. Sudoku Solver ★★ ★
18. Word Search ★★ 19. Word Search II ★★ ★ 20. Expression Add Operators ★★ 21.
Partition to K Equal Sum Subsets ★ 22. Matchsticks to Square 23. Split Array into
Fibonacci Sequence

Revision:

- Subsets II, Combination Sum II, N-Queens, Sudoku Solver, Word Search II

PHASE 3: GRAPH DOMINATION (Weeks 11-16)

WEEK 11 — Matrix as Graph

Topics:

- Matrix traversal as graph
- BFS on matrix
- DFS on matrix
- Multi-source BFS
- Obstacle handling

Problems (18):

1. Number of Islands ★★
2. Max Area of Island ★
3. Flood Fill ★
4. Word Search (revisit) ★★
5. Word Search II (revisit) ★★ ★
6. Surrounded Regions ★
7. Pacific Atlantic Water Flow ★★
8. Shortest Bridge ★
9. Shortest Path in Binary Matrix ★
10. Shortest Path with Obstacles Elimination ★★
11. Rotting Oranges ★★
12. Walls and Gates ★
13. As Far from Land as Possible
14. 01 Matrix ★
15. Minimum Knight Moves
16. Unique Paths III ★

- 17. Island Perimeter
- 18. Coloring A Border

Revision:

- Number of Islands, Rotting Oranges, Pacific Atlantic, Shortest Path
-

WEEK 12 — Graph BFS/DFS Fundamentals

Topics:

- Graph representations
- BFS traversal
- DFS traversal
- Connected components
- Graph coloring

Problems (20):

1. Clone Graph ★★
2. Number of Connected Components ★
3. Graph Valid Tree ★
4. All Paths From Source to Target
5. Find if Path Exists in Graph
6. Keys and Rooms ★
7. Reorder Routes to Make All Paths Lead to City Zero
8. Minimum Number of Vertices to Reach All Nodes
9. Find the Town Judge
10. Find Center of Star Graph
11. Count Unreachable Pairs of Nodes
12. Evaluate Division ★★
13. Accounts Merge (preview) ★★
14. Similar String Groups (preview)
15. Number of Provinces
16. Redundant Connection (preview) ★
17. Satisfiability of Equality Equations
18. Possible Bipartition ★
19. Is Graph Bipartite ★★
20. Detonate the Maximum Bombs

Revision:

- Clone Graph, Is Graph Bipartite, Evaluate Division
-

WEEK 13 — Topological Sort + Cycle Detection

[MOCK INTERVIEW #1]

Topics:

- Kahn's algorithm (BFS)
- DFS-based topological sort
- Cycle detection in directed graphs
- Cycle detection in undirected graphs
- DAG properties

Problems (15):

1. Course Schedule ★★
2. Course Schedule II ★★
3. Course Schedule IV
4. Alien Dictionary ★★
5. Find Eventual Safe States ★
6. Sequence Reconstruction
7. Parallel Courses ★
8. Sort Items by Groups Respecting Dependencies ★★
9. Minimum Height Trees ★★
10. Build a Matrix With Conditions
11. Longest Increasing Path in Matrix ★★
12. All Ancestors of a Node in DAG
13. Detect Cycle in Directed Graph (GFG)
14. Detect Cycle in Undirected Graph (GFG)
15. Loudest and Richest ★

Revision:

- Course Schedule I & II, Alien Dictionary, Find Eventual Safe States

MOCK INTERVIEW #1:

- 2 medium problems (1 from arrays/sliding window, 1 from stacks)
- Timed: 45 minutes total
- Practice verbal explanation

WEEK 14 — Union Find (Disjoint Set Union)

Topics:

- Union-Find data structure
- Path compression
- Union by rank/size
- Minimum Spanning Tree (Kruskal's)
- Dynamic connectivity

Problems (15):

1. Redundant Connection ★★
2. Redundant Connection II ★★
3. Number of Connected Components in Undirected Graph ★
4. Graph Valid Tree ★
5. Accounts Merge ★★
6. Most Stones Removed with Same Row or Column ★
7. Similar String Groups ★
8. Regions Cut By Slashes ★★
9. Friend Circles / Number of Provinces
10. Satisfiability of Equality Equations ★
11. Smallest String With Swaps ★
12. Min Cost to Connect All Points (Kruskal MST) ★★
13. Number of Operations to Make Network Connected ★
14. Minimize Malware Spread ★
15. Earliest Moment When Everyone Become Friends

Revision:

- Redundant Connection, Accounts Merge, Regions Cut By Slashes, Kruskal MST
-

WEEK 15 — Shortest Path Algorithms

Topics:

- Dijkstra's algorithm
- Bellman-Ford algorithm
- Floyd-Warshall algorithm
- 0-1 BFS
- Multi-source shortest path

Problems (15):

1. Network Delay Time (Dijkstra) ★★
2. Path With Minimum Effort (Dijkstra) ★★
3. Cheapest Flights Within K Stops (Bellman-Ford) ★★
4. Find the City With Smallest Number of Neighbors (Floyd-Warshall) ★
5. Shortest Path in Binary Matrix (BFS) ★
6. Minimum Cost to Make at Least One Valid Path (0-1 BFS) ★★
7. Swim in Rising Water ★★
8. Maze II ★
9. Word Ladder ★★
10. Word Ladder II ★★
11. The Maze
12. Shortest Path to Get All Keys ★★
13. Minimum Weighted Subgraph With Required Paths ★★

- 14. Reachable Nodes In Subdivided Graph ★★
- 15. Minimum Obstacle Removal to Reach Corner ★★

Revision:

- Network Delay, Path With Minimum Effort, Word Ladder, Cheapest Flights
-

WEEK 16 — Advanced Graph Patterns

Topics:

- Strongly connected components (Tarjan's/Kosaraju's)
- Bridges and articulation points
- Eulerian path/circuit
- Bipartite matching
- Network flow basics

Problems (12):

1. Critical Connections in Network (Bridges) ★★
2. Articulation Points (GFG)
3. Strongly Connected Components (Kosaraju's - GFG)
4. Strongly Connected Components (Tarjan's - GFG)
5. Reconstruct Itinerary (Eulerian Path) ★★
6. Valid Arrangement of Pairs (Eulerian Path) ★★
7. Graph Connectivity With Threshold ★
8. Checking Existence of Edge Length Limited Paths ★★
9. Maximum Bipartite Matching (GFG)
10. Maximum Flow (Ford-Fulkerson - GFG)
11. Count Subtrees With Max Distance Between Cities ★★
12. Number of Restricted Paths From First to Last Node ★★

Revision:

- Critical Connections, Reconstruct Itinerary, SCCs
-

PHASE 4: DP ELITE MASTERY (Weeks 17-22)

WEEK 17 — 1D Dynamic Programming

Topics:

- State definition
- Recurrence relation
- Base cases

- Space optimization
- 1D state transitions

Problems (15):

1. Fibonacci Number
2. Climbing Stairs ★
3. Min Cost Climbing Stairs ★
4. House Robber ★★
5. House Robber II ★
6. Delete and Earn ★
7. Jump Game ★★
8. Jump Game II ★★
9. Decode Ways ★★
10. Decode Ways II ★
11. Maximum Subarray (DP perspective) ★
12. Maximum Product Subarray ★★
13. Divisor Game
14. Perfect Squares ★★
15. Counting Bits ★

Revision:

- House Robber, Jump Game II, Decode Ways, Perfect Squares
-

WEEK 18 — Knapsack Dynamic Programming

Topics:

- 0/1 Knapsack
- Unbounded Knapsack
- Bounded Knapsack
- Subset sum variations
- Target sum patterns

Problems (15):

1. 0/1 Knapsack (GFG) ★★
2. Partition Equal Subset Sum ★★
3. Partition to K Equal Sum Subsets (revisit)
4. Target Sum ★★
5. Ones and Zeroes ★★
6. Last Stone Weight II ★
7. Coin Change ★★
8. Coin Change II ★★
9. Combination Sum IV (DP approach) ★★
10. Perfect Squares (Knapsack perspective)

11. Minimum Subset Sum Difference (GFG) ★
12. Count of Subset Sum (GFG)
13. Unbounded Knapsack (GFG) ★
14. Rod Cutting (GFG) ★
15. Minimum Cost For Tickets ★★

Revision:

- 0/1 Knapsack, Partition Equal Subset Sum, Coin Change I & II, Target Sum
-

WEEK 19 — 2D Dynamic Programming

Topics:

- 2D state definition
- Grid DP
- String matching DP
- Longest common subsequence variants
- Edit distance variations

Problems (15):

1. Unique Paths ★★
2. Unique Paths II ★
3. Unique Paths III (revisit)
4. Minimum Path Sum ★★
5. Triangle ★★
6. Maximal Square ★★
7. Longest Common Subsequence ★★
8. Longest Palindromic Subsequence ★★
9. Edit Distance ★★
10. Distinct Subsequences ★★
11. Interleaving String ★★
12. Regular Expression Matching ★★
13. Wildcard Matching ★★
14. Dungeon Game ★★
15. Cherry Pickup ★★

Revision:

- LCS, Edit Distance, Regular Expression Matching, Cherry Pickup
-

WEEK 20 — LIS (Longest Increasing Subsequence) Pattern

[MOCK INTERVIEW #2]

Topics:

- LIS with $O(n^2)$ DP
- LIS with $O(n \log n)$ binary search
- LIS variations
- Number of LIS
- Multi-dimensional LIS

Problems (15):

1. Longest Increasing Subsequence ★★★★★
2. Number of Longest Increasing Subsequence ★★
3. Longest Continuous Increasing Subsequence
4. Longest Bitonic Subsequence (GFG) ★
5. Maximum Sum Increasing Subsequence (GFG) ★
6. Increasing Triplet Subsequence ★
7. Russian Doll Envelopes ★★★★★
8. Maximum Length of Pair Chain ★★
9. Box Stacking (GFG) ★
10. Building Bridges (GFG)
11. Largest Divisible Subset ★★
12. Longest Arithmetic Subsequence ★★
13. Longest Arithmetic Subsequence of Given Difference ★
14. Wiggle Subsequence ★★
15. Delete Columns to Make Sorted III ★★

Revision:

- LIS (both approaches), Number of LIS, Russian Doll Envelopes

MOCK INTERVIEW #2:

- 2 medium problems (1 tree, 1 graph)
 - 1 easy problem (linked list)
 - Timed: 60 minutes total
-

WEEK 21 — Interval Dynamic Programming**Topics:**

- Interval merging
- Optimal binary search tree
- Matrix chain multiplication
- Palindrome partitioning
- Game theory on intervals

Problems (15):

1. Matrix Chain Multiplication (GFG) ★★★★★
2. Burst Balloons ★★★★★
3. Minimum Cost to Cut a Stick ★★★★★
4. Minimum Cost Tree From Leaf Values ★★★★★
5. Palindrome Partitioning II ★★★★★
6. Palindrome Partitioning III ★★★★★
7. Palindrome Partitioning IV ★★★★★
8. Strange Printer ★★★★★
9. Remove Boxes ★★★★★
10. Boolean Parenthesization (GFG) ★★★★★
11. Optimal Binary Search Tree (GFG) ★★★★★
12. Stone Game ★★★★★
13. Stone Game II ★★★★★
14. Stone Game III ★★★★★
15. Minimum Score Triangulation of Polygon ★★★★★

Revision:

- MCM, Burst Balloons, Palindrome Partitioning II, Remove Boxes

WEEK 22 — Bitmask DP + State Machine DP

Topics:

- Bitmask representation
- Traveling Salesman Problem
- Assignment problem
- State machine transitions
- Stock trading with states

Problems (15):

Bitmask DP:

1. Traveling Salesman Problem (GFG) ★★★★★
2. Assignment Problem (GFG) ★★★★★
3. Shortest Superstring ★★★★★
4. Partition to K Equal Sum Subsets (bitmask approach) ★★★★★
5. Smallest Sufficient Team ★★★★★
6. Maximum Students Taking Exam ★★★★★
7. Fair Distribution of Cookies ★★★★★
8. Minimum XOR Sum of Two Arrays ★★★★★
9. Number of Ways to Wear Different Hats ★★★★★
10. Find Minimum Cost to Reach Destination ★★★★★

State Machine DP: 11. Best Time to Buy and Sell Stock ★★★★★ 12. Best Time to Buy and Sell Stock II ★★★★★ 13. Best Time to Buy and Sell Stock III ★★★★★ 14. Best Time to Buy and Sell

Stock IV ★★★★★ 15. Best Time to Buy and Sell Stock with Cooldown ★★★★★ 16. Best Time to Buy and Sell Stock with Transaction Fee ★★

Additional (Optional):

- Paint House ★
- Paint House II ★★
- Paint Fence ★

Revision:

- TSP, Shortest Superstring, Stock III & IV, Stock with Cooldown
-

PHASE 5: ADVANCED PATTERNS (Weeks 23-27)

WEEK 23 — Trie Pattern

Topics:

- Trie implementation
- Prefix matching
- Word search with Trie
- Auto-complete systems
- Dictionary operations

Problems (12):

1. Implement Trie (Prefix Tree) ★★
2. Design Add and Search Words Data Structure ★★
3. Replace Words ★★
4. Word Search II (revisit with Trie) ★★★★★
5. Design Search Autocomplete System ★★★★★
6. Longest Word in Dictionary ★★
7. Map Sum Pairs ★
8. Prefix and Suffix Search ★★
9. Stream of Characters ★★
10. Palindrome Pairs ★★★★★
11. Concatenated Words ★★★★★
12. Maximum XOR of Two Numbers in Array ★★

Revision:

- Implement Trie, Word Search II, Design Autocomplete, Palindrome Pairs
-

WEEK 24 — Advanced String Algorithms

Topics:

- KMP (Knuth-Morris-Pratt)
- Rabin-Karp rolling hash
- Z-algorithm
- Manacher's algorithm
- String hashing

Problems (10):

1. Implement strStr() / Find Index of First Occurrence (KMP) ★★
2. Repeated Substring Pattern (KMP) ★
3. Shortest Palindrome (KMP) ★★★★★
4. Longest Happy Prefix (KMP/Z-algo) ★★
5. Repeated DNA Sequences (Rabin-Karp) ★★
6. Longest Duplicate Substring (Rabin-Karp + Binary Search) ★★★★★
7. Find All Anagrams in String (rolling hash) ★
8. Longest Palindromic Substring (Manacher's) ★★
9. Count Palindromic Substrings ★★
10. Distinct Echo Substrings ★★

Revision:

- KMP implementation, Shortest Palindrome, Longest Duplicate

Substring

WEEK 25 — Intervals + Greedy Algorithms

Topics:

- Interval merging
- Interval scheduling
- Greedy choice property
- Activity selection
- Job sequencing

Problems (15):

Intervals:

1. Merge Intervals ★★
2. Insert Interval ★★
3. Non-overlapping Intervals ★★
4. Meeting Rooms ★
5. Meeting Rooms II ★★
6. Minimum Number of Arrows to Burst Balloons ★★
7. Interval List Intersections ★★

8. Employee Free Time ★★★★★
9. My Calendar I ★
10. My Calendar II ★★
11. My Calendar III ★★
12. Divide Intervals Into Minimum Number of Groups ★★

Greedy: 13. Jump Game (greedy approach) ★★ 14. Jump Game II (greedy approach) ★★ 15. Gas Station ★★ 16. Candy ★★★★★ 17. Partition Labels ★★ 18. Minimum Platforms (GFG) ★ 19. Activity Selection (GFG) ★ 20. Job Sequencing (GFG) ★

Revision:

- Merge Intervals, Meeting Rooms II, Employee Free Time, Candy

WEEK 26 — Design Data Structures + Math

[MOCK INTERVIEW #3]

Topics:

- Cache design
- Data structure design
- Mathematical algorithms
- Number theory

Problems (18):

Design:

1. LRU Cache ★★★★★
2. LFU Cache ★★★★★
3. Design HashMap ★
4. Design HashSet ★
5. Design Twitter ★★
6. Design Browser History ★★
7. Design Hit Counter ★★
8. Design Search Autocomplete System (revisit) ★★★★★
9. Min Stack ★
10. Max Stack ★
11. Implement Queue using Stacks ★
12. Implement Stack using Queues ★
13. Design Circular Queue ★
14. Design Circular Deque ★

Math: 15. Pow(x, n) ★★ 16. Sqrt(x) ★ 17. Integer to Roman ★ 18. Roman to Integer ★ 19. Factorial Trailing Zeroes ★ 20. Count Primes (Sieve of Eratosthenes) ★★ 21. Happy Number ★ 22. Ugly Number II ★★ 23. Super Pow ★ 24. Nth Digit ★★ 25. Excel Sheet Column Number ★ 26. Excel Sheet Column Title ★

Revision:

- LRU Cache, LFU Cache, Design Twitter, Pow(x,n), Count Primes

MOCK INTERVIEW #3:

- 1 medium DP problem
 - 1 medium graph problem
 - 1 design problem
 - Timed: 75 minutes total
-

WEEK 27 — Bit Manipulation + Advanced Sorting

Topics:

- Bitwise operations
- XOR properties
- Bit tricks
- Advanced sorting techniques
- Counting sort, radix sort

Problems (12):

Bit Manipulation:

1. Single Number ★
2. Single Number II ★★
3. Single Number III ★★
4. Counting Bits (revisit) ★
5. Hamming Distance ★
6. Total Hamming Distance ★★
7. Reverse Bits ★
8. Number of 1 Bits ★
9. Power of Two ★
10. Power of Four ★
11. Bitwise AND of Numbers Range ★★
12. Maximum XOR of Two Numbers (Trie approach) ★★
13. Gray Code ★★
14. Subsets (bit manipulation approach) ★

Sorting: 15. Sort Colors (revisit) ★ 16. Sort List (revisit) ★★ 17. Pancake Sorting ★★
18. Count of Smaller Numbers After Self ★★★★★ 19. Reverse Pairs ★★★★★ 20. Count of Range Sum ★★★★★

Revision:

- Single Number II & III, Maximum XOR, Count of Smaller Numbers

PHASE 6: INTERVIEW MASTERY (Weeks 28-30)

WEEKS 28-29 — Intensive Mock Interviews

Structure:

- **3 mocks per week = 6 total mocks**
- Each mock: 2 problems in 60 minutes
- Mix of medium + hard
- Practice on Pramp, Interviewing.io, or with peers

Mock Distribution:

Mock 4-5 (Week 28):

- Arrays/Sliding Window + Trees
- Graphs + DP
- Backtracking + Binary Search

Mock 6-7 (Week 28):

- Linked List + Stack
- Heap + Greedy
- Design + Intervals

Mock 8-9 (Week 29):

- Mixed medium problems (full simulation)
- Hard graph + Hard DP
- System design basics (for senior roles)

Daily Practice (alongside mocks):

- Solve 2 random medium problems
- 1 hard problem every 2 days
- Review 5 old problems daily

Total Additional Problems: ~30 problems

WEEK 30 — Company-Specific Targeting

Google Focus:

- Graph algorithms (complex)
- DP (hard problems)

- Tree algorithms
- System design (distributed systems)

Google Problems (7):

1. Alien Dictionary ★★★★★
2. Longest Increasing Path in Matrix ★★
3. Word Ladder II ★★★★★
4. Cherry Pickup ★★★★★
5. Regular Expression Matching ★★★★★
6. Median of Two Sorted Arrays ★★★★★
7. Trapping Rain Water II ★★★★★

Meta Focus:

- DP optimization
- Backtracking
- Graph + BFS/DFS
- Design problems

Meta Problems (7):

1. Edit Distance ★★★★★
2. Word Break II ★★★★★
3. Binary Tree Maximum Path Sum ★★★★★
4. Accounts Merge ★★
5. Design Twitter ★★
6. Lowest Common Ancestor ★★
7. Subarray Sum Equals K ★

Amazon Focus:

- Medium problems with optimal solutions
- Arrays, strings, trees, graphs
- Practical OOP design
- Leadership principles prep

Amazon Problems (7):

1. Two Sum variants ★
2. Merge K Sorted Lists ★
3. LRU Cache ★★★★★
4. Number of Islands ★★
5. Course Schedule II ★★
6. Top K Frequent Elements ★
7. Sliding Window Maximum ★★★★★

Apple Focus:

- Clean, bug-free code

- Edge case handling
- Fundamentals mastery

Apple Problems (5):

1. Validate BST ★★
2. Merge Intervals ★★
3. Reverse Linked List variants ★
4. Longest Substring Without Repeating ★
5. Binary Tree Level Order ★

Netflix Focus:

- System design heavy
- Scalability considerations
- Advanced data structures

Netflix Problems (4):

1. Design Rate Limiter
2. Design Distributed Cache
3. LFU Cache ★★★
4. Time-based Key-Value Store ★★

Total: 30 company-specific problems



FINAL PROBLEM COUNT SUMMARY

Phase	Weeks	Problems
Foundation	0-6	125
Trees + Backtracking	7-10	78
Graph Domination	11-16	100
DP Elite Mastery	17-22	90
Advanced Patterns	23-27	77
Interview Mastery	28-30	60
TOTAL	30	~530



MASTER REVISION SYSTEM

Daily Revision (Non-Negotiable)

- **Morning (15 min):** Review pattern notes from yesterday
- **Evening (15 min):** Dry-run 1 previously solved problem on paper

Spaced Repetition Schedule

After solving any problem:

- **Day 3:** Re-solve (time: ≤ 15 min)
- **Day 7:** Re-solve (time: ≤ 20 min)
- **Day 21:** Re-solve (time: ≤ 25 min)
- **Day 60:** Final review

Weekly Revision (Every Sunday)

- **Morning (2 hours):**
 - Re-solve 5 hardest problems from the week
 - Write pattern summaries from memory
- **Afternoon (1 hour):**
 - Pattern flash cards review
 - Identify patterns from 10 new problem statements (don't solve)

Monthly Mega-Revision (Last day of each month)

- **30-problem sprint:** Top problems from that month
 - **Pattern recognition test:** 20 problems, identify pattern only
 - **Mock interview:** Full simulation
-

SUCCESS METRICS & CHECKPOINTS

After Week 6 (Foundation Complete):

- ✓ Solve 90% of easy problems in ≤ 10 min
- ✓ Solve 70% of medium (arrays/strings) in ≤ 25 min
- ✓ Recognize sliding window/two-pointer patterns instantly
- ✓ Can implement all 5 stack patterns from memory

After Week 10 (Trees + Backtracking Complete):

- ✓ Solve 85% of medium problems in ≤ 25 min
- ✓ Can write tree recursion without hesitation
- ✓ Backtracking template memorized
- ✓ Mock interview pass rate $\geq 50\%$

After Week 16 (Graphs Complete):

- ✓ Solve 80% of medium in ≤ 20 min
- ✓ Solve 40% of hard in ≤ 45 min
- ✓ Can model any graph problem in ≤ 3 min
- ✓ Mock interview pass rate $\geq 65\%$

After Week 22 (DP Complete):

- ✓ Solve 90% of medium in ≤ 20 min
- ✓ Solve 60% of hard in ≤ 40 min
- ✓ Can derive DP recurrence in ≤ 5 min
- ✓ Mock interview pass rate $\geq 75\%$

After Week 30 (Interview Ready):

- ✓ Solve 95% of medium in ≤ 20 min
- ✓ Solve 75% of hard in ≤ 40 min
- ✓ Pattern recognition in ≤ 60 sec
- ✓ Mock interview pass rate $\geq 85\%$
- ✓ Can explain solution while coding fluently
- ✓ Bug-free first submission rate $\geq 80\%$

CRITICAL EXECUTION RULES

Non-Negotiables:

1. **No skipping weeks or reordering** — Sequence is pedagogically designed
2. **Time every single problem** — Interview simulation is critical
3. **Verbal explanation before coding** — Communication practice
4. **Write time/space complexity before coding** — Forces analytical thinking
5. **Complete all revision cycles** — Retention > volume
6. **One full rest day per week** — Sustainability prevents burnout
7. **30-minute rule:** If stuck >30 min, read hints (not solution)
8. **No solution copy-paste** — Type every line yourself

Red Flags (Stop & Reassess If You See These):

- ✗ Solving >30% hard before Week 16
- ✗ Looking at solutions before 25 min effort
- ✗ Skipping revision cycles
- ✗ Not timing problems
- ✗ Solving problems in random order
- ✗ Mock pass rate <50% after Week 20
- ✗ Can't explain solution out loud

Green Flags (You're On Track):

- ✓ Can identify pattern in <90 sec
 - ✓ Write brute force first, then optimize
 - ✓ Explain complexity without hesitation
 - ✓ Code compiles on first try 70%+ of time
 - ✓ Completing weekly targets
 - ✓ Revision cycles executed consistently
-

PROBLEM-SOLVING FRAMEWORK (Use for Every Problem)

Step 1: Clarify (2-3 min)

- Understand input/output
- Ask about constraints (array size, value ranges)
- Clarify edge cases

Step 2: Example (2-3 min)

- Walk through 2-3 test cases manually
- Include edge cases

Step 3: Approach (5-10 min)

- **Identify the pattern** (this should take <60 sec with practice)
- Explain brute force approach first
- Optimize: explain why optimization works
- State time/space complexity

Step 4: Code (15-20 min)

- Write clean, readable code
- Use meaningful variable names
- Add comments for complex logic

Step 5: Test (5-10 min)

- Dry run with example
- Test edge cases
- Check for off-by-one errors

Step 6: Optimize (if time)

- Discuss space optimization

- Alternative approaches
- Trade-offs

Total Time Target: 45 minutes for medium, 60 minutes for hard



RESOURCE RECOMMENDATIONS

Primary Platform:

- **LeetCode Premium** (essential for company-specific lists)

Supplementary:

- **NeetCode.io** — Excellent video explanations for Blind 75
- **AlgoMonster** — Pattern-based learning
- **Educative.io: Grokking the Coding Interview** — Pattern recognition
- **InterviewBit** — Structured path

Video Resources:

- **NeetCode (YouTube)** — Problem walkthroughs
- **Back To Back SWE** — Deep algorithm explanations
- **William Fiset** — Graph algorithms
- **Abdul Bari** — Algorithm fundamentals
- **Tushar Roy** — DP explanations

Books (Optional but Recommended):

- **"Cracking the Coding Interview"** by Gayle McDowell
- **"Elements of Programming Interviews"** (Python/Java/C++)
- **"Introduction to Algorithms"** (CLRS) — Reference only

Mock Interview Platforms:

- **Pramp** — Free peer interviews
 - **Interviewing.io** — Anonymous interviews with engineers
 - **LeetCode Mock Assessments**
 - **Exponent** — FAANG-focused
-



MISTAKE JOURNAL TEMPLATE

For every problem you get wrong or struggle with:

Problem: [Name + LeetCode #]

Date: [Date]

Time Taken: [Actual time]

WHY I FAILED:

- [Specific reason: didn't recognize pattern, implementation bug, etc.]

PATTERN MISSED:

- [Which pattern: sliding window, DP, etc.]

KEY INSIGHT:

- [The "aha" moment that unlocks the problem]

SIMILAR PROBLEMS:

1. [Problem name]

2. [Problem name]

3. [Problem name]

RECURRENCE RELATION (if DP):

- [State definition + transition]

TIME/SPACE COMPLEXITY:

- Time: $O(?)$

- Space: $O(?)$

REVISIT DATES:

- Day 3: []

- Day 7: []

- Day 21: []

WEEKLY SCHEDULE TEMPLATE

Monday-Friday (Core Solving Days):

Morning (2 hours):

- Review yesterday's patterns (15 min)

- Solve 2-3 new problems (90 min)

- Document in mistake journal (15 min)

Evening (2 hours):

- Solve 1-2 new problems (60 min)

- Revision: re-solve 1 old problem (30 min)

- Dry-run 1 problem on paper (30 min)

Saturday (Heavy Practice Day):

Morning (3 hours):

- Solve 3-4 new problems
- Focus on harder problems from the week

Afternoon (2 hours):

- Timed mock (if scheduled)
- OR solve 2 company-specific problems

Sunday (Revision & Planning Day):

Morning (2 hours):

- Re-solve 5 hardest problems from week
- Write pattern summaries from memory

Afternoon (1 hour):

- Pattern flash card review
- Plan next week
- Update progress tracker

Evening:

- COMPLETE REST (no coding)



PROGRESS TRACKING SPREADSHEET

Create a spreadsheet with these columns:

Date	Problem	Difficulty	Time	Pattern	First Attempt?	Revisit 1	Revisit 2	Revisit 3	Notes
					✓/✗	Date	Date	Date	

Track:

- **Total problems solved**
 - **Weekly completion rate**
 - **Average time per difficulty**
 - **Mock interview scores**
 - **Pattern mastery checklist**
-



FINAL OUTCOME EXPECTATIONS

If you execute this plan with **90%+ consistency**:

Technical Outcomes:

- ✓ Top 1-3% DSA capability globally
- ✓ 500+ LeetCode problems solved deeply
- ✓ All major patterns mastered
- ✓ Can solve most mediums in 15-20 min
- ✓ Can solve most hards in 35-45 min
- ✓ Pattern recognition instantaneous

Interview Outcomes:

- ✓ 80-90% MAANG interview pass rate
- ✓ Multiple offers likely
- ✓ Confidence in any coding interview
- ✓ Can negotiate from position of strength

Career Outcomes:

- ✓ MAANG-level compensation
 - ✓ Career trajectory acceleration
 - ✓ Problem-solving skills for life
 - ✓ Foundation for staff/principal engineering
-



EXECUTION BEGINS NOW

This is not theory. This is your **battle plan**.

Next Steps:

1. ✓ Save this document
 2. ✓ Set up LeetCode account
 3. ✓ Create progress tracking spreadsheet
 4. ✓ Block 4-6 hours daily on calendar
 5. ✓ Join Discord/Reddit study groups
 6. ✓ **Start Week 0, Day 1, Problem 1 TODAY**
-



WEEK 0, DAY 1 STARTS NOW

Problem 1: Maximum Subarray (Kadane's Algorithm)

Say **"GO"** and I'll teach you:

- The exact Kadane pattern
- How to recognize it in 30 seconds
- The template to memorize forever
- 5 variations using the same logic

This is your path to MAANG.

Are you ready? Type "GO" to begin. 🚀

Tab 2

EXCELLENT IDEA — Here's Why You're 100% Right

Adding a **dedicated Matrix/2D Array week** is **strategically brilliant**. Here's why:

Why Matrix Deserves Its Own Week

1. Matrix Problems Are EVERYWHERE in MAANG

- **Amazon:** 20-25% of interviews include matrix problems
- **Google:** Matrix + graph hybrid problems are favorites
- **Meta:** Grid-based DP and traversal problems are common
- **Apple:** Clean matrix manipulation tests fundamentals

2. Matrix Skills Transfer to Multiple Patterns

- **Graph algorithms** (matrix-as-graph)
- **2D DP** (grid DP)
- **Backtracking** (word search, N-queens)
- **Sliding window** (2D sliding window)
- **Binary search** (search in 2D matrix)

3. Gap in Original Plan

The original schedule had matrix problems scattered:

- Week 0: Spiral Matrix, Set Matrix Zeroes
- Week 11: Matrix-as-Graph
- Week 19: 2D DP on grids

Result: No dedicated mastery of matrix fundamentals before advanced applications.

REVISED SCHEDULE: Matrix Week Added

Option 1: Insert Matrix Week Early (Recommended)

WEEK 0 — Arrays + Sorting

WEEK 0.5 — Matrix/2D Arrays ★ NEW

WEEK 1 — Sliding Window + Two Pointers

...rest continues

Rationale:

- Build matrix foundation BEFORE using it in graphs/DP
 - Matrix manipulation is foundational like arrays
 - Better pedagogical flow
-

Option 2: Insert After Arrays Week

WEEK 0 — Arrays + Sorting

WEEK 1 — Matrix/2D Arrays ★ NEW

WEEK 2 — Sliding Window + Two Pointers

WEEK 3 — Hashing + Prefix Sum + Strings

...rest continues



WEEK 0.5/1 — MATRIX/2D ARRAYS MASTERY

Duration: 7 days

Problems: 20-25

Goal: Complete matrix pattern mastery before advanced topics

Topics Covered:

Day 1-2: Matrix Traversal Fundamentals

- Row-wise, column-wise traversal
- Diagonal traversal
- Spiral traversal
- Zigzag traversal
- Snake pattern
- Boundary traversal

Day 3-4: Matrix Manipulation

- Rotate matrix (90°, 180°, 270°)
- Transpose matrix
- In-place modifications
- Set zeroes
- Fill techniques

Day 5: Matrix Search

- Search in row-wise sorted matrix
- Search in row-column sorted matrix
- Binary search on matrix
- Peak element in 2D

Day 6: Matrix Math & Properties

- Matrix multiplication
- Determinant (if needed)
- Prefix sum 2D
- Range sum query 2D

Day 7: Mixed Practice & Revision

- Hardest problems from week
- Pattern recognition drill
- Timed practice

Complete Problem List (25 Problems):

A) Traversal Patterns (7 problems)

1. **Spiral Matrix** ★★
 - Pattern: Layer-by-layer traversal
 - Key: Boundary management
2. **Spiral Matrix II** ★
 - Generate spiral matrix
 - Key: Direction control
3. **Diagonal Traverse** ★★
 - Diagonal zigzag pattern
 - Key: Direction changes on boundaries
4. **Print Matrix in Snake Pattern** (GFG)
 - Alternate left-right, right-left
 - Key: Row parity
5. **Print Matrix in Diagonal Pattern** (GFG)
 - All diagonals sequentially
 - Key: Sum of indices
6. **Boundary Traversal of Matrix** (GFG)
 - Print outer boundary
 - Key: Edge handling

7. Zigzag (Wave) Traversal (GFG)

- Column-wise zigzag
 - Key: Column parity
-

B) Matrix Rotation & Transformation (6 problems)

8. Rotate Image (90° clockwise) ★★

- In-place rotation
- Key: Transpose + reverse rows
- **CRITICAL PATTERN**

9. Rotate Image 180° (variation)

- Two rotations OR reverse rows + columns

10. Transpose Matrix

- Convert rows to columns
- Key: Swap $[i][j]$ with $[j][i]$

11. Set Matrix Zeroes ★★

- Set entire row/col to zero
- Key: In-place marking using first row/col
- **AMAZON FAVORITE**

12. Reshape Matrix ★

- Convert $m \times n$ to $r \times c$
- Key: Linear mapping

13. Flip Image ★

- Horizontal flip + invert
 - Key: Two-pointer in each row
-

C) Matrix Search (5 problems)

14. Search a 2D Matrix ★★

- Sorted matrix (treat as 1D array)
- Key: Binary search with row $\times$ col conversion
- **GOOGLE FAVORITE**

15. Search a 2D Matrix II ★★

- Row-wise & column-wise sorted
- Key: Start from top-right or bottom-left
- **META FAVORITE**

16. Find Peak Element II ★★

- 2D peak (greater than all 4 neighbors)
- Key: Binary search on columns

17. Kth Smallest Element in Sorted Matrix ★★

- Row & column sorted
- Key: Binary search on value range OR min-heap

18. Count Negative Numbers in Sorted Matrix ★

- Sorted rows & columns
 - Key: Staircase search
-

D) Matrix Math & Prefix Sum (4 problems)

19. Range Sum Query 2D - Immutable ★★

- Submatrix sum queries
- Key: 2D prefix sum
- **CRITICAL PATTERN FOR DP**

20. Range Sum Query 2D - Mutable ★★

- With updates
- Key: 2D Binary Indexed Tree (advanced)

21. Matrix Block Sum ★

- Sum of submatrix around each cell
- Key: 2D prefix sum

22. Valid Sudoku ★★

- Check sudoku validity
 - Key: Row/col/box validation
-

E) Matrix Island Patterns (Preview for Week 11) (3 problems)

23. Island Perimeter ★

- Count perimeter of island
- Key: Count edges

24. Max Area of Island ★★

- Find largest island
- Key: DFS with area counting
- **Preview of matrix-as-graph**

25. Number of Closed Islands ★★

- Islands not touching boundary
 - Key: DFS with boundary check
-

Learning Outcomes:

After this week, you will:

- ✓ Master all matrix traversal patterns
 - ✓ Solve rotation/transformation in-place instantly
 - ✓ Recognize when to use 2D prefix sum
 - ✓ Know staircase search pattern for sorted matrices
 - ✓ Convert matrix indices $\leftrightarrow$ linear indices fluently
 - ✓ Handle boundary conditions without bugs
 - ✓ Build foundation for matrix-as-graph (Week 11)
 - ✓ Build foundation for 2D DP (Week 19)
-

Why This Sequencing Works

Before Matrix Week:

- Week 0: Arrays (1D foundation)

Matrix Week:

- Master 2D fundamentals

After Matrix Week:

- Week 1: Sliding Window (can now do 2D sliding window)
- Week 11: Matrix-as-Graph (already know traversals)
- Week 19: 2D DP (already know grid navigation)
- Week 21: Interval DP (some use 2D tables)

Result: Compound learning effect, each week builds on previous



UPDATED 31-WEEK SCHEDULE

Week

Topic

Problem
s

0	Arrays + Sorting	15
0.5	Matrix/2D Arrays ★	25
1	Sliding Window + Two Pointers	15
2	Hashing + Prefix Sum + Strings	20
3	Linked List	20
4	Stack Patterns	25
5	Heap/PQ	15
6	Binary Search	20
7	Tree DFS/BFS	20
8	Path Recursion + Tree DP	20
9	BST	15
10	Backtracking	23
11	Matrix as Graph	18
...	(rest continues)	...

New Total Duration: 31 weeks (7.75 months)

New Total Problems: ~555 problems

Matrix Week Daily Breakdown

Day 1: Traversal Basics

- Spiral Matrix ★★
- Spiral Matrix II
- Boundary Traversal
- Wave/Zigzag Traversal
- **Goal:** Master all traversal directions

Day 2: Advanced Traversal

- Diagonal Traverse ★★
- Diagonal Pattern
- Snake Pattern
- **Practice:** Draw patterns on paper first
- **Goal:** Direction change logic

Day 3: Rotation & Transpose

- Rotate Image 90° ★★ (CRITICAL)
- Transpose Matrix
- Flip Image
- Rotate 180°
- **Goal:** In-place transformations

Day 4: Matrix Modification

- Set Matrix Zeroes ★★ (AMAZON FAVORITE)
- Reshape Matrix
- **Practice:** O(1) space solutions
- **Goal:** In-place algorithms

Day 5: Matrix Search

- Search 2D Matrix ★★
- Search 2D Matrix II ★★ (META FAVORITE)
- Find Peak Element II
- Count Negative Numbers
- **Goal:** Binary search on 2D

Day 6: Prefix Sum & Math

- Range Sum Query 2D ★★ (CRITICAL FOR DP)
- Matrix Block Sum
- Valid Sudoku ★★
- Kth Smallest in Sorted Matrix
- **Goal:** 2D prefix sum mastery

Day 7: Mixed Practice + Preview

- Island Perimeter
- Max Area of Island ★★
- Number of Closed Islands
- **Timed practice:** Re-solve 5 hardest
- **Goal:** Pattern recognition speed

Key Matrix Patterns to Memorize

Pattern 1: Spiral Traversal Template

```
def spiralOrder(matrix):  
    result = []  
    top, bottom = 0, len(matrix) - 1
```

```

left, right = 0, len(matrix[0]) - 1

while top <= bottom and left <= right:
    # Right
    for col in range(left, right + 1):
        result.append(matrix[top][col])
    top += 1

    # Down
    for row in range(top, bottom + 1):
        result.append(matrix[row][right])
    right -= 1

    # Left (if still valid)
    if top <= bottom:
        for col in range(right, left - 1, -1):
            result.append(matrix[bottom][col])
        bottom -= 1

    # Up (if still valid)
    if left <= right:
        for row in range(bottom, top - 1, -1):
            result.append(matrix[row][left])
        left += 1

return result

```

Pattern 2: In-Place 90° Rotation

```

def rotate(matrix):
    n = len(matrix)

    # Transpose
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Reverse each row
    for row in matrix:
        row.reverse()

```

Pattern 3: Staircase Search (Search 2D Matrix II)

```

def searchMatrix(matrix, target):
    if not matrix:
        return False

```

```
row, col = 0, len(matrix[0]) - 1 # Start top-right
```

```
while row < len(matrix) and col >= 0:
```

```
    if matrix[row][col] == target:
```

```
        return True
```

```
    elif matrix[row][col] > target:
```

```
        col -= 1 # Move left
```

```
    else:
```

```
        row += 1 # Move down
```

```
return False
```

Pattern 4: 2D Prefix Sum

```
def buildPrefixSum(matrix):
```

```
    m, n = len(matrix), len(matrix[0])
```

```
    prefix = [[0] * (n + 1) for _ in range(m + 1)]
```

```
    for i in range(1, m + 1):
```

```
        for j in range(1, n + 1):
```

```
            prefix[i][j] = matrix[i-1][j-1] + \
```

```
                prefix[i-1][j] + \
```

```
                prefix[i][j-1] - \
```

```
                prefix[i-1][j-1]
```

```
    return prefix
```

```
def rangeSum(prefix, r1, c1, r2, c2):
```

```
    # Sum of submatrix from (r1,c1) to (r2,c2)
```

```
    return prefix[r2+1][c2+1] - \
```

```
        prefix[r1][c2+1] - \
```

```
        prefix[r2+1][c1] + \
```

```
        prefix[r1][c1]
```



Final Verdict

Adding dedicated Matrix week = SMART MOVE

Benefits:

- ✓ Stronger foundation for graphs & DP
- ✓ Faster problem recognition
- ✓ Fewer bugs in 2D problems
- ✓ Confidence in 25% more interview problems
- ✓ Better prepared for Amazon/Google matrix-heavy interviews

Cost:

- ⚠️ +1 week total time (31 weeks vs 30)
- ⚠️ +25 problems

ROI:

📊 **EXTREMELY HIGH** — Matrix skills pay dividends throughout entire schedule

UPDATED EXECUTION PLAN

Choose Your Sequencing:

Option A (Recommended): Matrix Week 0.5

Week 0: Arrays → Week 0.5: Matrix → Week 1: Sliding Window

Pros: Earlier foundation, better for graphs/DP later

Option B: Matrix Week 1

Week 0: Arrays → Week 1: Matrix → Week 2: Sliding Window

Pros: More time to absorb arrays before adding 2D

YOUR UPDATED ROADMAP

I'll now revise the complete 31-week schedule with Matrix Week included.

Say "FINALIZE" and I'll give you:

- ✅ Complete 31-week schedule with Matrix Week integrated
- ✅ All 25 matrix problems with difficulty ratings
- ✅ Day-by-day breakdown for Matrix Week
- ✅ Matrix pattern cheat sheet
- ✅ Updated total problem count

Ready to finalize? Type "FINALIZE" 🚀

Tab 3

```
if (amount==0) {  
    return 0;  
}  
Arrays.sort(coins);  
int count=0,nums=1;  
  
for(int i=0;i<coins.length;i++){  
  
    if (amount>=coins[i]) {  
  
        nums=amount%coins[i];  
        count+=nums;  
        amount=amount-nums*price[i];  
  
    }  
  
}  
return amount;
```